



8.2.2 Toward practical descent methods ¶

8.2.2 Toward practical descent methods, Part 1

fit width

Even though matrices are often highly sparse, a major part of the cost of solving $Ax = b$ via descent methods is in the matrix-vector multiplication (a cost that is proportional to the number of nonzeros in the matrix). For this reason, reducing the number of these is an important part of the design of the algorithm.

Homework 8.2.2.1. Let

$$\begin{aligned}x^{(k+1)} &= x^{(k)} + \alpha_k p^{(k)} \\r^{(k)} &= b - Ax^{(k)} \\r^{(k+1)} &= b - Ax^{(k+1)}\end{aligned}$$

Show that

$$r^{(k+1)} = r^{(k)} - \alpha_k Ap^{(k)}.$$

▼ Solution

$$\begin{aligned}r^{(k+1)} &= b - Ax^{(k+1)} \\&= \langle r^{(k)} = b - Ax^{(k)} \rangle \\r^{(k+1)} &= r^{(k)} + Ax^{(k)} - Ax^{(k+1)} \\&= \langle \text{rearrange, factor} \rangle \\r^{(k+1)} &= r^{(k)} - A(x^{(k+1)} - x^{(k)}) \\&= \langle x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)} \rangle \\r^{(k+1)} &= r^{(k)} - \alpha_k Ap^{(k)}\end{aligned}$$

Alternatively:

$$\begin{aligned}r^{(k+1)} &= b - Ax^{(k+1)} \\&= \langle x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)} \rangle \\r^{(k+1)} &= b - A(x^{(k)} + \alpha_k p^{(k)}) \\&= \langle \text{distribute} \rangle \\r^{(k+1)} &= b - Ax^{(k)} - \alpha_k Ap^{(k)} \\&= \langle \text{definition of } r^{(k)} \rangle \\r^{(k+1)} &= r^{(k)} - \alpha_k Ap^{(k)}\end{aligned}$$

8.2.2 Toward practical descent methods, Part 2

fit width

With the insights from this last homework, we can reformulate our basic descent method into one with only one matrix-vector multiplication, as illustrated in [Figure 8.2.2.1](#).

Given : $A, b, x^{(0)}$	Given : $A, b, x^{(0)}$	Given : $A, b, x^{(0)}$
$r^{(0)} := b - Ax^{(0)}$	$r^{(0)} := b - Ax^{(0)}$	$r^{(0)} := b - Ax^{(0)}$
$k := 0$	$k := 0$	$k := 0$
while $r^{(k)} \neq 0$	while $r^{(k)} \neq 0$	while $r^{(k)} \neq 0$
$p^{(k)} = \text{next direction}$	$p^{(k)} := \text{next direction}$	$p^{(k)} := \text{next direction}$
$\alpha_k := \frac{p^{(k)T} r^{(k)}}{p^{(k)T} A p^{(k)}}$	$\alpha_k := \frac{p^{(k)T} r^{(k)}}{p^{(k)T} A p^{(k)}}$	$\alpha_k := \frac{p^{(k)T} r^{(k)}}{p^{(k)T} q^{(k)}}$
$x^{(k+1)} := x^{(k)} + \alpha_k p^{(k)}$	$x^{(k+1)} := x^{(k)} + \alpha_k p^{(k)}$	$x^{(k+1)} := x^{(k)} + \alpha_k p^{(k)}$
$r^{(k+1)} := b - Ax^{(k+1)}$	$r^{(k+1)} := r^{(k)} - \alpha_k A p^{(k)}$	$r^{(k+1)} := r^{(k)} - \alpha_k q^{(k)}$
$k := k + 1$	$k := k + 1$	$k := k + 1$
endwhile	endwhile	endwhile

Figure 8.2.2.1. Left: Basic descent method from last unit. Middle: Minor modification that recasts the computation of the residual $r^{(k+1)}$ as an update of the previous residual $r^{(k)}$. Right: modification that reduces the number of matrix-vector multiplications by introducing temporary vector $q^{(k)}$.

Homework 8.2.2.2. For loops in the algorithm in [Figure 8.2.2.1](#) (Right), count the number of matrix-vector multiplications, dot products, and "axpy" operations (not counting the cost of determining the next descent direction).

▼ [Solution](#)

$q^{(k)} := Ap^{(k)}$	1 mvmult
$\alpha_k := \frac{p^{(k)T} r^{(k)}}{p^{(k)T} q^{(k)}}$	2 dot products
$x^{(k+1)} := x^{(k)} + \alpha_k p^{(k)}$	1 axpy
$r^{(k+1)} := r^{(k)} - \alpha_k q^{(k)}$	1 axpy

Total: 1 mvmults, 2 dot products, 2 axpys

8.2.2 Toward practical descent methods, Part 3

fit width

We finish our discussion regarding basic descent methods by observing that we don't need to keep the history of vectors, $x^{(k)}$, $p^{(k)}$, $r^{(k)}$, $q^{(k)}$, and scalar α_k that were computed as long as they are not needed to compute the next search direction, leaving us with the algorithm

```

Given :  $A, b, x$ 
 $r := b - Ax$ 
while  $r \neq 0$ 
   $p := \text{next direction}$ 
   $q := Ap$ 
   $\alpha := \frac{p^T r}{p^T q}$ 
   $x := x + \alpha p$ 
   $r := r - \alpha q$ 
endwhile

```

Figure 8.2.2.2. The algorithm from [Figure 8.2.2.1](#) (Right) storing only the most current vectors and scalar.